

# RELATÓRIO TÉCNICO: ATIVIDADE EXPLORATÓRIA DE MICROCONTROLADORES

Pontifícia Universidade Católica de Minas Gerais

Instituto de Ciências Exatas e Informática

Laboratório de Arquitetura de Computadores

Este relatório apresenta a análise de viabilidade, seleção de arquitetura, justificativa técnica, **análise de custo-benefício (viabilidade econômica)** e caracterização de hardware para seis projetos distintos de sistemas embarcados. Conforme o item 2 do enunciado, cada arquitetura é escolhida pelo critério de ser a mais adequada **técnica e economicamente**, seguindo o princípio do professor de que "o *subdimensionamento gera falhas catastróficas; o superdimensionamento gera desperdício financeiro*" — ou seja, nem uma bazuca para matar uma mosca, nem um estilingue para parar um tanque. A análise é baseada estritamente nos datasheets fornecidos no diretório do projeto.

## Tabela Resumo das Escolhas

| Projeto | Nome do Projeto                  | Viabilidade             | Arquitetura Selecionada             | Tipo de Memória       | Núcleos | Custo Unit. Aprox.* |
|---------|----------------------------------|-------------------------|-------------------------------------|-----------------------|---------|---------------------|
| 1       | IoT – Rede de Sensores Agrícolas | MCU Clássico (SoC)      | <b>ESP32</b> (ex: ESP32-D0WD-V3)    | Harvard Modificada    | 2       | ~R\$ 15–25          |
| 2       | Robótica – Controlador de Eixo   | MCU Clássico            | <b>STM32F103C8</b> (Cortex-M3)      | Harvard Modificada    | 1       | ~R\$ 10–20          |
| 3       | Automação – Gateway e Central    | Exige Processador/SBC   | <b>Raspberry Pi 3 Model B</b>       | Von Neumann (Sistema) | 4       | ~R\$ 300–450        |
| 4       | Biomedicina – Oxímetro Portátil  | MCU Clássico            | <b>ATtiny85</b> (AVR 8-bit)         | Harvard Pura          | 1       | ~R\$ 3–8            |
| 5       | Drone – Controlador de Voo       | MCU Clássico            | <b>STM32F103C8</b> (Cortex-M3)      | Harvard Modificada    | 1       | ~R\$ 10–20          |
| 6       | Áudio – Fone de Ouvido com ANC   | MCU Especializado / SoC | <b>ESP32</b> (Análise de Trade-off) | Harvard Modificada    | 2       | ~R\$ 15–25          |

\* Valores aproximados de ordem de grandeza (preço unitário do chip/módulo em varejo/baixo volume), apenas para fins de comparação de custo-benefício. Variam conforme fornecedor, quantidade e câmbio.

## PROJETO 1: Internet das Coisas (IoT) – Rede de Sensores Agrícolas

### 1. Análise de Viabilidade

O problema pode ser resolvido puramente com microcontroladores clássicos, mais especificamente com um **SoC (System-on-Chip) que integre conectividade sem fio**. O uso de um microprocessador ou SBC (como a Raspberry Pi 3) é inviável, pois o consumo elétrico de um sistema operacional completo e de um processador em escala de GHz inviabilizaria a alimentação por bateria por meses (consumo de centenas de mA vs. a meta de microamperes).

### 2. Seleção de Arquitetura

A arquitetura selecionada é o **SoC ESP32** (ex: ESP32-D0WD-V3).

- Alternativa considerada: ESP8266EX.

- **Motivo da escolha:** O ESP8266EX possui uma restrição severa de memória RAM disponível para o usuário. De acordo com a seção 3.1 do seu datasheet (*Memory*), em modo Station conectado a um roteador, o espaço máximo de RAM disponível para o código do usuário é **menor que 50 KB** ("RAM size < 50 KB"). Como o projeto exige cerca de 50 KB livres apenas para o código do usuário e a pilha Wi-Fi, o ESP8266EX fica no limite ou insuficiente. Além disso, o consumo em *Deep-sleep* do ESP8266EX a 2.5V é de exatamente **20 µA**, não oferecendo margem de segurança para o requisito de ser inferior a 20 µA. O **ESP32** resolve esses problemas oferecendo **520 KB de SRAM** interna e um consumo em *Deep-sleep* de apenas **10 µA** (mantendo a memória e periféricos do RTC ligados), além de possuir hardware I2C nativo.

### 3. Justificativa Técnica

- **Conectividade Wi-Fi e Stack TCP/IP:** O ESP32 possui rádio Wi-Fi 802.11 b/g/n nativo integrado com pilha TCP/IP/IPv4 completa em hardware/SDK.
- **Gerenciamento de Energia:** O consumo de 10 µA em *Deep-sleep* atende com folga o limite de 20 µA a 2.5V (seção 4.3 do datasheet do ESP32). Isso permite que o dispositivo passe 99% do tempo dormindo, acordando rapidamente via timer do RTC para ler os sensores e transmitir os dados, garantindo meses de autonomia de bateria.
- **Memória:** Com 520 KB de SRAM, há memória abundante para hospedar a aplicação do usuário (~50 KB) simultaneamente com as pilhas de Wi-Fi e segurança (TLS/TCP).
- **Periféricos:** Possui barramento I2C integrado para leitura direta e rápida de sensores digitais de temperatura/umidade.

### 4. Análise de Custo-Benefício (Viabilidade Econômica)

- **Preço:** O ESP32 custa cerca de **R\$ 15–25** por módulo, valor irrisório frente ao que entrega (rádio Wi-Fi + TCP/IP + 520 KB SRAM + dois núcleos integrados em um único chip).
- **Comparação com a alternativa (ESP8266 ~R\$ 8–15):** O ESP8266 é uns R\$ 5–10 mais barato, mas seria uma economia falsa: sua RAM < 50 KB fica no limite do projeto e o *Deep-sleep* de 20 µA não dá margem ao requisito. Pagar esses poucos reais a mais pelo ESP32 compra 10× mais RAM e folga de energia que garantem meses de autonomia — **o melhor valor por real gasto**.
- **Por que não superdimensionar (SBC):** Usar uma Raspberry Pi 3 (~R\$ 300–450, mais de 15× o custo) seria desperdício financeiro clássico ("bazuca para matar mosca"): além de inviabilizar a alimentação por bateria, multiplicaria o custo unitário em uma rede com centenas de nós sensores. O ESP32 é, portanto, o ponto ótimo técnico-econômico.

### 5. Características do Dispositivo Escolhido

- **Tipo de Memória: Harvard Modificada.** O processador Xtensa LX6 possui barramentos separados de instruções (iBus) e dados (dBus), permitindo acessos paralelos para maximizar a performance. No entanto, mapeia os espaços de endereço lógicos de forma unificada no mapa de memória física.
- **Núcleos de Processamento: Dual-Core.** O ESP32 possui dois processadores Xtensa 32-bit LX6 (denominados PRO\_CPU e APP\_CPU), embora possa ser configurado para rodar tarefas do sistema em um núcleo e a aplicação do usuário no outro.

## PROJETO 2: Robótica – Controlador de Eixo para Braço Robótico

### 1. Análise de Viabilidade

O problema pode e deve ser resolvido utilizando **microcontroladores clássicos de 32 bits de alto desempenho**. O uso de uma SBC com Linux (como a Raspberry Pi 3) é fortemente contraindicado devido à falta de determinismo (jitter) provocada pelo agendador do sistema operacional, o que pode atrasar o ciclo de controle de malha fechada (PID) e causar instabilidade ou danos mecânicos na junta robótica.

### 2. Seleção de Arquitetura

A arquitetura selecionada é a baseada no núcleo ARM Cortex-M3 de 32 bits, especificamente o microcontrolador **STM32F103C8**.

- *Alternativa considerada:* PIC32MX5XX/6XX/7XX.
- *Motivo da escolha:* O STM32F103C8 opera a **72 MHz** (faixa de 72 a 80 MHz exigida) e possui o timer avançado **TIM1 (Advanced-control timer)** de 16 bits. Este timer é projetado especificamente para controle de motores, fornecendo 6 saídas PWM complementares com geração de **tempo morto (dead-time)** programável em hardware e uma **entrada de emergência Break (BKIN)** que desarma as saídas instantaneamente em caso de falha mecânica ou elétrica, sem depender de interrupção via software. O PIC32MX possui módulos Output Compare, mas estes não implementam geração de dead-time complementar com entrada break de emergência de forma nativa e integrada na mesma flexibilidade que o TIM1 do STM32. Adicionalmente, o ADC do STM32F103C8 é de **12 bits a 1 Msps**, fornecendo mais precisão (4096 níveis) que o ADC de 10 bits (1024 níveis) do PIC32MX, essencial para feedback preciso de posição e corrente.

### 3. Justificativa Técnica

- **Processamento:** O núcleo ARM Cortex-M3 de 32 bits rodando a 72 MHz atinge 1.25 DMIPS/MHz, efetuando multiplicação em ciclo único e divisão por hardware, garantindo a execução ultra-rápida do algoritmo PID de controle de eixo.
- **Periféricos Específicos:** O timer TIM1 garante a comutação segura de pontes H (evitando curto-circuitos através do dead-time) e proteção contra sobrecorrente via pino Break. Os dois ADCs de 12 bits de 1 Msps realizam leituras instantâneas dos sensores analógicos de corrente e posição.
- **Conectividade:** Possui periférico de hardware dedicado para comunicação **CAN 2.0B Ativo**, permitindo comunicação determinística em ambientes industriais ruidosos.
- **Tempo Real:** Arquitetura altamente determinística com baixíssima latência de interrupção (graças ao NVIC).

### 4. Análise de Custo-Benefício (Viabilidade Econômica)

- **Preço:** O STM32F103C8 custa cerca de **R\$ 10–20**. É um dos 32-bits de melhor relação custo/desempenho do mercado — tanto que se tornou padrão industrial (a popular placa "Blue Pill").
- **Comparação com a alternativa (PIC32MX ~R\$ 25–40):** Aqui a escolha técnica e a econômica apontam para o mesmo lado: o STM32F103C8 é **mais barato E mais capaz** que o PIC32MX para esta aplicação. O PIC32MX custa ~2× mais e ainda assim não traz o dead-time complementar com entrada *Break* nativo (essencial para a ponte H do motor) nem o ADC de 12 bits. Pagar mais por menos recurso seria injustificável.
- **Por que não subdimensionar:** Um MCU de 8 bits custaria centavos, mas não fecha o laço PID de *hard real-time* em tempo — o subdimensionamento aqui geraria falha catastrófica (dano mecânico na junta). O STM32 é o menor custo que atende com segurança o requisito crítico.

### 5. Características do Dispositivo Escolhido

- **Tipo de Memória: Harvard Modificada.** O Cortex-M3 possui barramentos separados de instruções (I-Code) e dados (D-Code/System Bus) para permitir buscas de instruções e acessos à SRAM em paralelo, embora utilize um mapa de endereçamento de 4 GB unificado.
- **Núcleos de Processamento: Single-Core** (1 núcleo ARM Cortex-M3).

## PROJETO 3: Automação Residencial – Gateway e Central Inteligente

### 1. Análise de Viabilidade

Este projeto **exige obrigatoriamente um microprocessador / Single Board Computer (SBC)**. É impossível resolvê-lo com microcontroladores clássicos de 8 ou 32 bits (como o STM32 ou AVR), pois o processamento de voz em linguagem natural, a decodificação de vídeo em tempo real (1080p30 H.264) e a

renderização de uma interface gráfica complexa para saída em TV exigem poder computacional na casa dos GHz, centenas de megabytes de memória RAM e suporte a um sistema operacional completo (como o Linux) para gerenciar as pilhas de software de alto nível.

## 2. Seleção de Arquitetura

A arquitetura selecionada é a **Raspberry Pi 3 Model B** (baseada no SoC Broadcom BCM2837).

- *Motivo da escolha:* O dispositivo atende perfeitamente a todos os requisitos do projeto:
  - **Processador:** Broadcom BCM2837 com CPU **Quad-Core ARM Cortex-A53 de 1.2 GHz e 64 bits** (atende a especificação de 1.2 GHz Quad-Core 64-bit).
  - **Memória:** **1 GB de RAM LPDDR2** (atende o requisito de pelo menos 1 GB de RAM).
  - **Vídeo:** Coprocessador multimídia **VideoCore IV** que suporta decodificação por hardware de H.264 a 1080p30.
  - **Conectividade:** Conectividade Wi-Fi 802.11n e Bluetooth 4.1 clássico/LE integrados, além de porta Ethernet 10/100.
  - **Periféricos:** Saídas de vídeo nativas **HDMI** e interface de display **DSI**.
  - **Sistema Operacional:** Suporte nativo e otimizado a sistemas Linux completos (como o Raspberry Pi OS).

## 3. Justificativa Técnica

Microcontroladores clássicos possuem pouca RAM (geralmente < 1 MB) e velocidades de clock baixas (geralmente < 200 MHz), inviabilizando qualquer codec de decodificação H.264 Full HD ou mecanismos locais/nuvem de reconhecimento de voz. A Raspberry Pi 3 fornece a largura de banda de memória necessária (1 GB), processamento multicore para paralelizar comandos de voz e interface gráfica, além de aceleradores gráficos dedicados para renderização em TV via HDMI sem sobrecarregar a CPU.

## 4. Análise de Custo-Benefício (Viabilidade Econômica)

- **Preço:** A Raspberry Pi 3 Model B custa cerca de **R\$ 300–450** — duas ordens de grandeza acima de um MCU. **Aqui o custo alto é justificado, não desperdício:** este é o caso em que o requisito (Linux + 1 GB RAM + decodificação H.264 1080p30 + GUI em HDMI) está acima do que qualquer MCU consegue, então pagar por capacidade que será de fato usada é a decisão economicamente correta.
- **Custo-benefício de montar uma alternativa discreta:** Tentar replicar essas funções com chips separados (CPU de aplicação + codec de vídeo dedicado + DRAM + controladora HDMI + rádios Wi-Fi/BT) elevaria o custo de materiais (BOM) e, principalmente, o custo e o tempo de engenharia muito acima dos R\$ 300–450 da SBC pronta. A RPi 3 integra tudo isso e ainda traz o ecossistema de software do Linux.
- **Por que não superdimensionar mais ainda:** Subir para um mini-PC x86 ou uma RPi de geração superior aumentaria o custo sem benefício, pois a RPi 3 já atende exatamente (1.2 GHz quad-core, 1 GB, VideoCore IV). Ela é o **piso econômico** que satisfaz o requisito.

## 5. Características do Dispositivo Escolhido

- **Tipo de Memória:** **Von Neumann** ao nível de sistema. O processador Broadcom BCM2837 armazena dados e instruções na mesma memória RAM externa física compartilhada (1 GB LPDDR2) e utiliza barramentos de sistema compartilhados. (Embora cada núcleo Cortex-A53 possua caches L1 de instrução e dados separados - característica do tipo Harvard para ganho de desempenho -, a arquitetura de memória externa do sistema é unificada Von Neumann).
- **Núcleos de Processamento:** **Multicore (Quad-Core)** (4 núcleos físicos ARM Cortex-A53).

# PROJETO 4: Equipamentos Biomédicos – Oxímetro Portátil Ultra-compacto

## 1. Análise de Viabilidade

Este projeto pode ser resolvido perfeitamente com **microcontroladores clássicos de 8 bits de ultra-baixo consumo**. O uso de processadores de 32 bits ou SBCs seria um superdimensionamento desastroso, gerando um custo inviável para um wearable descartável/portátil, além de aumentar o tamanho físico e drenar rapidamente a bateria.

## 2. Seleção de Arquitetura

A arquitetura selecionada é a baseada no AVR de 8 bits, especificamente o microcontrolador **ATtiny85**.

- *Alternativas consideradas:* ATmega328P, PIC18F4550.
- *Motivo da escolha:*
  - **Footprint físico:** O oxímetro exige espaço minúsculo. O ATtiny85 é oferecido em encapsulamento SOIC ou PDIP de apenas **8 pinos** (largura de 0.150" na versão SOIC), sendo ideal para wearables. O ATmega328P (28 pinos) e o PIC18F4550 (28/40 pinos) são fisicamente grandes demais e encarecem o projeto.
  - **Memória:** O ATtiny85 possui exatamente **8 KB de Flash** e **512 bytes de EEPROM** interna de alta retenção de dados (seção de memória do datasheet), encaixando-se perfeitamente no limite máximo estipulado pelo projeto.
  - **Periféricos e Consumo:** Possui ADC de 10 bits nativo e o modo de baixo ruído **ADC Noise Reduction Mode**, que desliga o clock da CPU e de I/O digitais durante a amostragem analógica para evitar ruídos de comutação. Adicionalmente, possui consumo de apenas **0.1 µA a 1.8V** em modo *Power-down* (com WDT desligado), atendendo ao requisito de 0.1 µA de consumo em repouso.

## 3. Justificativa Técnica

O projeto exige extrema economia de espaço, energia e custo. O ATtiny85 minimiza a contagem de pinos para 8, reduzindo a complexidade da PCB. A presença de 512 bytes de EEPROM é crucial para armazenar tabelas de calibração da leitura de saturação que precisam persistir após o desligamento do aparelho. O modo *ADC Noise Reduction* garante que a conversão analógica de 10 bits dos fotodiodos refletores de luz infravermelha e vermelha do oxímetro seja livre de ruído digital de alta frequência do clock do processador. O consumo de 0.1 µA em standby garante uma vida útil de anos para o dispositivo funcionando com uma bateria tipo botão (coin cell).

## 4. Análise de Custo-Benefício (Viabilidade Econômica)

- **Preço:** O ATtiny85 custa **poucos reais** (~R\$ 3–8). Este é o exemplo perfeito do princípio do professor: para uma tarefa simples de 8 bits, o MCU "custa centavos e consome frações de miliwatts".
- **Comparação com as alternativas:** O ATmega328P (~R\$ 10–15) e o PIC18F4550 (~R\$ 15–25) custam mais, ocupam muito mais espaço (28/40 pinos) e encarecem a PCB sem entregar nada que o projeto precise — recursos a mais que viram custo morto. O ATtiny85 entrega exatamente os 8 KB de Flash e 512 B de EEPROM exigidos, pelo menor preço e menor footprint.
- **Por que não superdimensionar:** Usar um STM32 (32 bits) ou uma SBC neste wearable seria o "superdimensionamento desastroso" descrito no relatório — multiplicaria o custo, o tamanho e o consumo num produto que tende a ser descartável/de alto volume, onde cada centavo no custo unitário importa. O ATtiny85 é, de longe, a **opção mais economicamente viável**.

## 5. Características do Dispositivo Escolhido

- **Tipo de Memória: Harvard Pura.** O núcleo AVR de 8 bits possui memórias física e logicamente separadas para programa (Flash) e dados (SRAM/EEPROM), com barramentos independentes de dados de 8 bits e barramento de instrução de 16 bits. Isso permite acesso simultâneo à instrução e à memória de dados na mesma oscilação de clock.
- **Núcleos de Processamento: Single-Core** (1 núcleo AVR 8-bit).

## PROJETO 5: Robótica Móvel e VANT – Controlador de Voo para Drone

### 1. Análise de Viabilidade

O problema pode e deve ser resolvido utilizando **microcontroladores clássicos de 32 bits de tempo real**. Sistemas embarcados como controladores de voo exigem taxas de atualização críticas e determinísticas (400 Hz ou mais). O uso de SBCs (como Raspberry Pi com Linux comum) é inadequado como controlador principal, pois a latência de interrupção do sistema operacional Linux de propósito geral (GPOS) pode ultrapassar milissegundos devido a troca de contextos, paginação de memória e desabilitação de interrupções, causando instabilidade fatal na aeronave.

### 2. Seleção de Arquitetura

A arquitetura selecionada é a baseada no ARM Cortex-M3 de 32 bits, especificamente o microcontrolador **STM32F103C8** (clock de 72 MHz).

- *Alternativas consideradas no contexto da diretriz de análise:* PIC32MX (MIPS32 a 80 MHz), ATmega2560 / ATmega328P (8 bits a 16 MHz), e Raspberry Pi 3 (SBC Linux a 1.2 GHz).

#### **Avaliação de Latência de Interrupções e Desempenho (Diretriz de Análise)**

##### 1. MCUs de 8 bits operando a 16 MHz (ex: ATmega328P / ATmega2560):

- *Latência:* Baixa latência teórica de hardware (~4 a 5 ciclos de clock = ~250 ns a 312 ns). Contudo, o salvamento de contexto (registradores) deve ser feito inteiramente via software na rotina ISR (gerando dezenas de ciclos extras de escrita na pilha, totalizando > 2 µs).
- *Desempenho:* A limitação severa é a CPU de 8 bits sem instruções de multiplicação acelerada (MAC) ou operações de ponto flutuante em ciclo único. Calcular o algoritmo PID de atitude (Pitch, Roll, Yaw) e aplicar fusão de sensores (Filtros de Kalman ou Complementares) em variáveis de 32 bits consome milhares de ciclos de instrução, estourando a janela de tempo de 2.5 ms do loop de 400 Hz.

##### 2. SBCs rodando Linux a 1.2 GHz (ex: Raspberry Pi 3):

- *Latência:* Altamente variável (com jitter de dezenas de microsegundos a milissegundos). Por rodar um SO de tempo compartilhado com memória virtual, a resposta a interrupções físicas de pinos (GPIO) não é determinística. Qualquer atraso pontual de 2 ms no controle de atitude causará a queda do drone.

##### 3. 32-bit MCUs (Cortex-M3 vs. MIPS32):

- **MIPS32 (PIC32MX a 80 MHz):** Possui pipeline eficiente e capacidade matemática, mas a resposta a interrupções requer rotinas de software complexas para salvar os registradores de uso geral na pilha, a menos que use "Shadow Register Sets" (bancos de registradores de sombra), recurso presente apenas para interrupções específicas de alta prioridade. A latência geral é menos determinística que no Cortex-M3.
- **Cortex-M3 (STM32F103C8 a 72 MHz):** Integra o **NVIC (Nested Vectored Interrupt Controller)**. O NVIC realiza o salvamento de contexto (registradores R0-R3, R12, LR, PC, xPSR) **automaticamente em hardware** em exatamente **12 ciclos de clock** (166.7 ns a 72 MHz). O retorno da interrupção leva 10 ciclos. Possui os recursos de *tail-chaining* (reduz a latência para 6 ciclos entre interrupções sequenciais) e *late-arriving* (processa uma interrupção de maior prioridade recém-chegada sem desperdiçar ciclos desempilhando a anterior).

Portanto, o **STM32F103C8 (Cortex-M3)** é tecnicamente superior para controle de voo, garantindo que as leituras do sensor inercial (IMU) e os cálculos de atitude aconteçam sempre dentro da janela temporal estrita.

### 3. Justificativa Técnica

- **Velocidade de Processamento:** Os 72 MHz da CPU Cortex-M3 com instruções **MAC (Multiply-Accumulate)** dedicadas executam em tempo recorde os algoritmos de controle de atitude e estabilização de voo.

- **Periféricos de Tempo Real:** Hardware SPI e I2C de alta velocidade integrados para leitura rápida da IMU (giroscópio/acelerômetro). Temporizadores de 16 bits para gerar sinais PWM (ou protocolos rápidos como DShot/Oneshot) de controle de velocidade para os ESCs dos motores.
- **NVIC Determinístico:** Garante jitter nulo ou desprezível no tempo de processamento do laço de controle.

#### 4. Análise de Custo-Benefício (Viabilidade Econômica)

- **Preço: ~R\$ 10–20.** Não por acaso, controladores de voo comerciais de baixo custo historicamente padronizaram justamente o STM32F1 — é o ponto em que custo e desempenho determinístico se encontram.
- **Comparação com as alternativas:** Um ATmega de 8 bits (~R\$ 10–15) custaria quase o mesmo, mas é um **subdimensionamento fatal**: não consegue calcular o PID de atitude com fusão de sensores dentro da janela de 2.5 ms (400 Hz) — economia que custaria a queda do drone. O PIC32MX (~R\$ 25–40) é mais caro e tem resposta a interrupções menos determinística que o NVIC do Cortex-M3. Logo, gastar mais no PIC32 traria pior resultado.
- **Por que não superdimensionar:** Uma SBC (RPi, ~R\$ 300–450) custaria mais de 15× e, pior, introduziria *jitter* não determinístico do Linux — custo maior por um resultado inadequado. O STM32F103C8 é o **menor custo que garante o tempo real crítico**, sendo a melhor relação custo-benefício.

#### 5. Características do Dispositivo Escolhido

- **Tipo de Memória: Harvard Modificada.** Possui barramentos de instruções (I-Code) e dados (D-Code) separados, operando em paralelo sobre o mapa unificado de memória física de 32 bits.
- **Núcleos de Processamento: Single-Core** (1 núcleo ARM Cortex-M3).

## PROJETO 6: Processamento de Áudio – Fones de Ouvido com ANC

### 1. Análise de Viabilidade

Este projeto pode ser resolvido com **SoCs microcontrolados especializados em processamento de sinais de áudio e conectividade sem fio**. Um processador/SBC do tipo Raspberry Pi é fisicamente grande demais para fones Intra-auriculares e consome energia em excesso. Microcontroladores tradicionais de 8 bits (AVR, PIC18) não possuem poder computacional para processamento digital de sinais (DSP) em frações de milissegundo, barramentos de áudio digital (I2S) ou rádio Bluetooth.

### 2. Seleção de Arquitetura

O único dispositivo presente na lista que possui rádio Bluetooth integrado e periférico I2S é o **ESP32**.

- **Nota Crítica:** Embora o ESP32 atenda tecnicamente à capacidade matemática e de periféricos, há um **forte trade-off de consumo de energia** que deve ser analisado.

#### Análise Rigorosa de Trade-off (Diretriz de Análise)

##### 1. Latência de Ciclo de Máquina:

- O ESP32 roda a **240 MHz**, o que resulta em um período de ciclo de máquina de apenas **4,16 ns**. O processador Xtensa LX6 possui instruções DSP (como MAC de ciclo único e registradores de janela). Isso permite ler amostras de áudio dos microfones externos via I2S, aplicar filtros digitais de resposta ao impulso finito (FIR/IIR) para inverter a fase e emitir a onda de anti-ruído via I2S/DAC em **sub-milissegundos**, atendendo à restrição de latência crítica do ANC.

##### 2. Consumo de Energia (O Gargalo):

- De acordo com a Tabela 5-4 do datasheet do ESP32 (*Current Consumption Depending on RF Modes*), o consumo de corrente ativa do rádio e da CPU transmitindo ou recebendo pacotes de RF (como áudio via Bluetooth A2DP) é de **aproximadamente 100 mA a 240 mA** (transmitindo com

potência máxima a POUT = +19.5 dBm consome 240 mA).

- Baterias típicas de fones de ouvido TWS (do tipo botão/coin cell) possuem capacidade entre **50 mAh e 80 mAh**.
- Operando continuamente com corrente média de ~100-150 mA, o fone TWS descarregaria em **menos de 30 minutos de uso**, tornando o ESP32 comercialmente inviável para fones TWS de prateleira.

### Conclusão do Trade-off e Solução Comercial Real

Embora o **ESP32** seja selecionado a partir dos arquivos fornecidos por ser o único com Bluetooth + I2S + DSP, um engenheiro de arquitetura deve apontar que um produto comercial real utilizaria um **SoC de Áudio Bluetooth dedicado e de ultra-baixo consumo** (como as famílias Qualcomm QCC51xx ou BES2300). Esses SoCs especializados contêm processadores DSP dedicados que realizam o processamento de ANC em hardware com baixíssimo consumo de corrente (< 5 mA a 10 mA em modo ativo de streaming Bluetooth + ANC), permitindo que a bateria de 50 mAh dure mais de 6 horas.

### 3. Justificativa Técnica

O ESP32 justifica-se unicamente pela presença do hardware de rádio Bluetooth integrado, periféricos de áudio digital **I2S** nativos para conexão direta a CODECs ou microfones MEMS de áudio digital, e a alta frequência de clock (240 MHz) do seu processador dual-core de 32 bits, capaz de processar os algoritmos complexos de ANC sem gerar atrasos de fase perceptíveis (latência inferior a frações de milissegundo).

### 4. Análise de Custo-Benefício (Viabilidade Econômica)

- **O preço do chip engana:** O ESP32 é barato (~R\$ 15–25), o que à primeira vista sugere ótimo custo-benefício. Porém, custo-benefício de um *produto* não é só o preço do chip — é o custo total de uma solução que **funcione**.
- **O custo escondido (autonomia):** Como demonstrado na análise de trade-off, o consumo de ~100–240 mA do rádio Bluetooth do ESP32 esgota uma bateria TWS de 50–80 mAh em menos de 30 minutos. Um fone que dura meia hora é **comercialmente inviável**: o "barato" sai caro em devoluções, reputação e inviabilidade do produto.
- **A escolha economicamente correta no mundo real:** Um SoC de áudio Bluetooth dedicado (Qualcomm QCC51xx, BES2300) custa um pouco mais por unidade (faixa de ~R\$ 15–40), mas seu DSP de ultra-baixo consumo (< 5–10 mA com BT+ANC) entrega 6+ horas de autonomia. **Pagar um pouco mais por um chip que torna o produto vendável é o melhor custo-benefício** — exatamente o tipo de trade-off que o enunciado pede. Dentre os datasheets fornecidos, porém, o ESP32 é a única opção tecnicamente capaz, e por isso é a selecionada, com esta ressalva econômica explícita.

### 5. Características do Dispositivo Escolhido

- **Tipo de Memória: Harvard Modificada.** Barramentos físicos independentes para instruções e dados com mapa lógico compartilhado.
- **Núcleos de Processamento: Dual-Core** (2 processadores Xtensa LX6 independentes).

---

*Fim do Relatório Técnico.*